

Architecture Microservices

1. De Enterprise-scale IT à Web-scale IT



Mouloud Menceur
mouloud.menceur@gmail.com

De Enterprise-scale IT à Web-scale IT (*)

« **Large cloud services providers** such as Amazon, Google, Facebook, etc., are **re-inventing** the way IT in which IT services can be delivered. »

Construire des systèmes **continuellement évolutifs** à l'échelle du Web avec **des niveaux de performance et de résilience jamais atteints**

Introduction

- **Problématique**
 - Construire des applications d'entreprise...
 - Fiables et sécurisées
 - Disponibles et tolérantes aux pannes
 - Supportant la montée en charge (*scalable*)
 - Transactionnelles
 - Administrables
 - Extensibles et réutilisables
 - Etc.
 - ... plus rapidement et pour moins cher !!

(*) http://web.archive.org/web/20200813115008/https://blogs.gartner.com/cameron_haight/2013/05/16/enter-web-scale-it/

Web-scale IT

- Cloud Computing - Virtualisation des infrastructures - Conteneurisation
- DevOps - automatisation de l'infrastructure - Continuous Delivery
- Organisation et processus agiles - Features teams - software craftsmanship
- Passage à l'échelle (*scalabilité*) - Design for failure

Web-scale IT

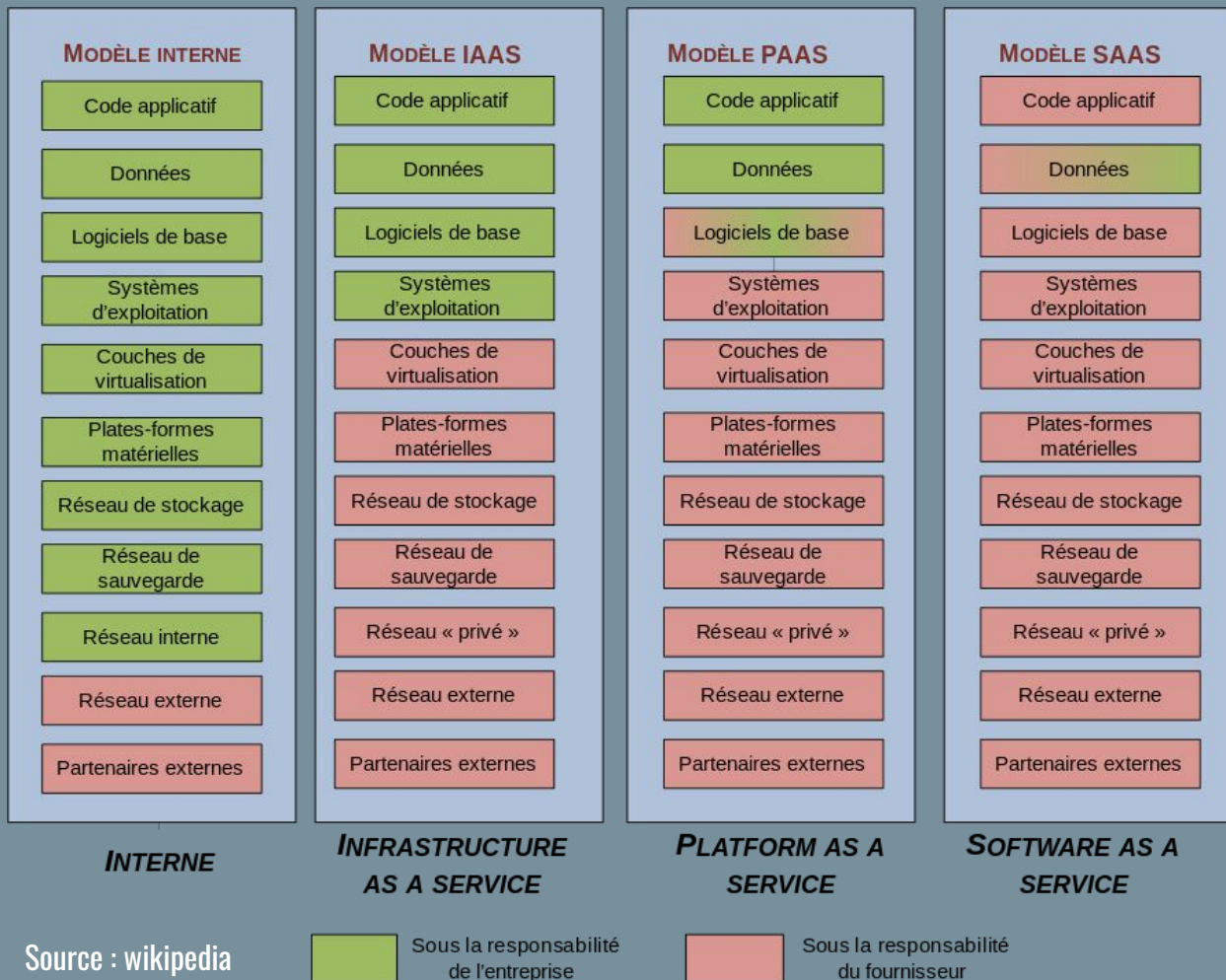
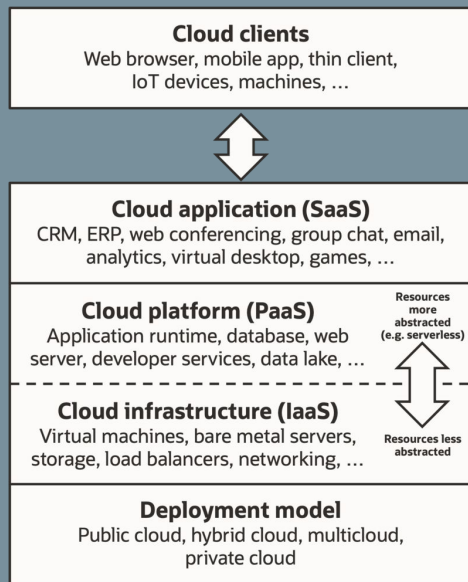
Cloud Computing - Virtualisation des infrastructures - Conteneurisation

“Le cloud computing est la **mise à disposition de ressources informatiques à la demande via Internet** , avec une tarification en fonction de votre utilisation. Au lieu d'acheter, de posséder et de gérer des serveurs et des centres de données physiques, **vous pouvez accéder à votre guise aux services technologiques, tels que la puissance de calcul, le stockage et les bases de données, d'un fournisseur cloud** ”

[AWS](#)

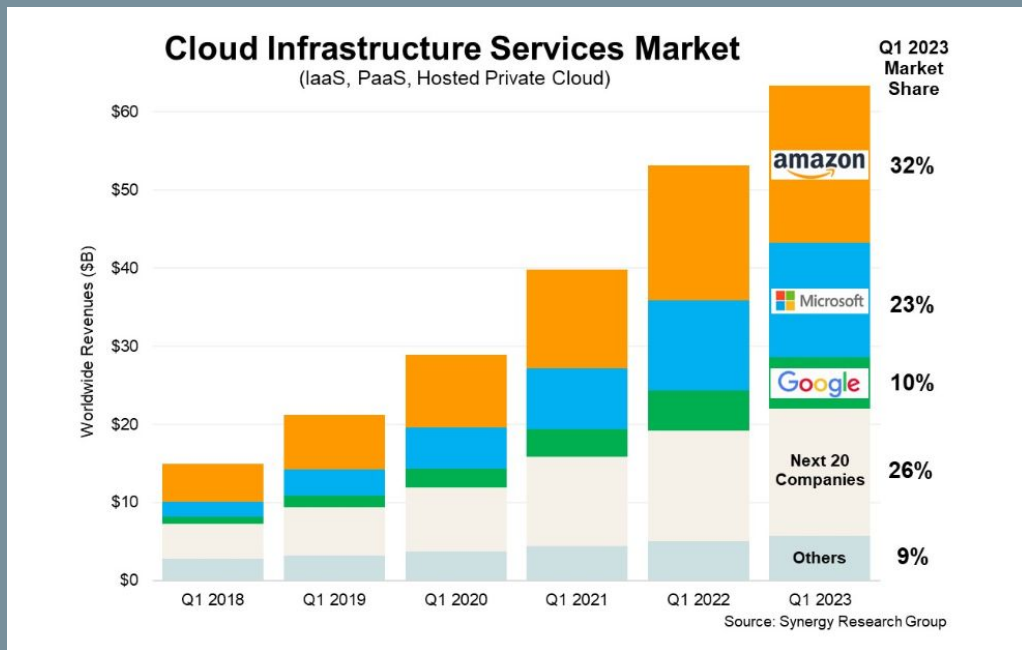
Web-scale IT

Cloud Computing - ...



Web-scale IT

Cloud Computing - Virtualisation des infrastructures - Conteneurisation



Google Cloud

Web-scale IT

Cloud Computing - **Virtualisation des infrastructures** - Conteneurisation

“Virtualization refers to the act of creating a virtual (rather than actual) version of something, including **virtual computer hardware platforms, storage devices, and computer network resources** [...] **Hardware virtualization refers to the creation of a virtual machine** that acts like a real computer with an operating system. Software executed on these virtual machines is separated from the underlying hardware resources” Wikipedia

Web-scale IT

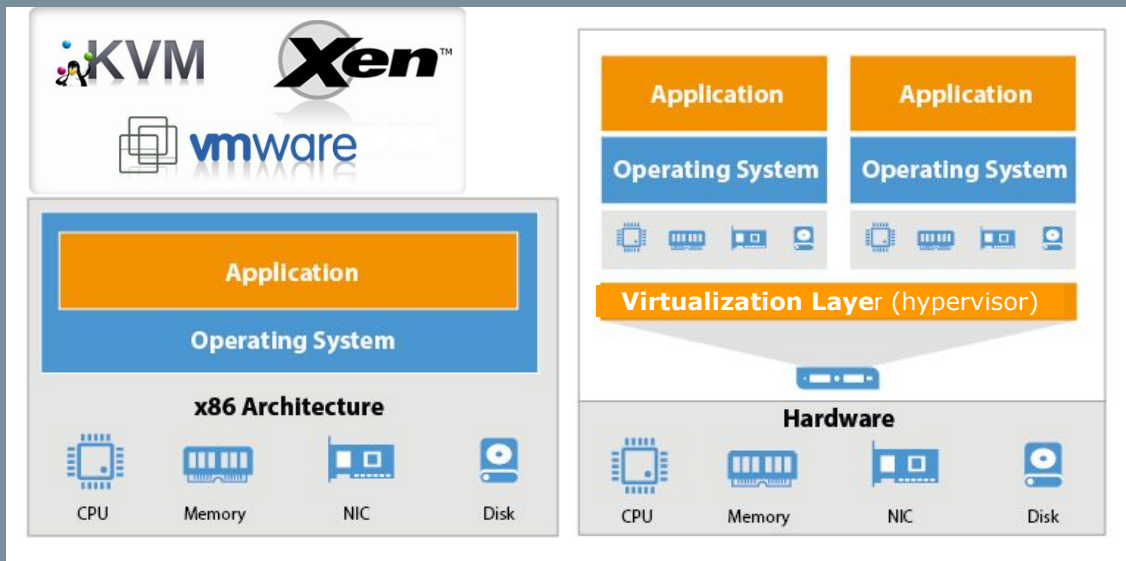
Cloud Computing - **Virtualisation des infrastructures** - Conteneurisation

Avantages

- Optimisation des ressources
- Flexibilité / Scalabilité
- Isolation / Sécurité
- Gestion/automatisation: déploiement, backup...

Inconvénients

- Performances
- Administration / gestion
- Cout initial et licences



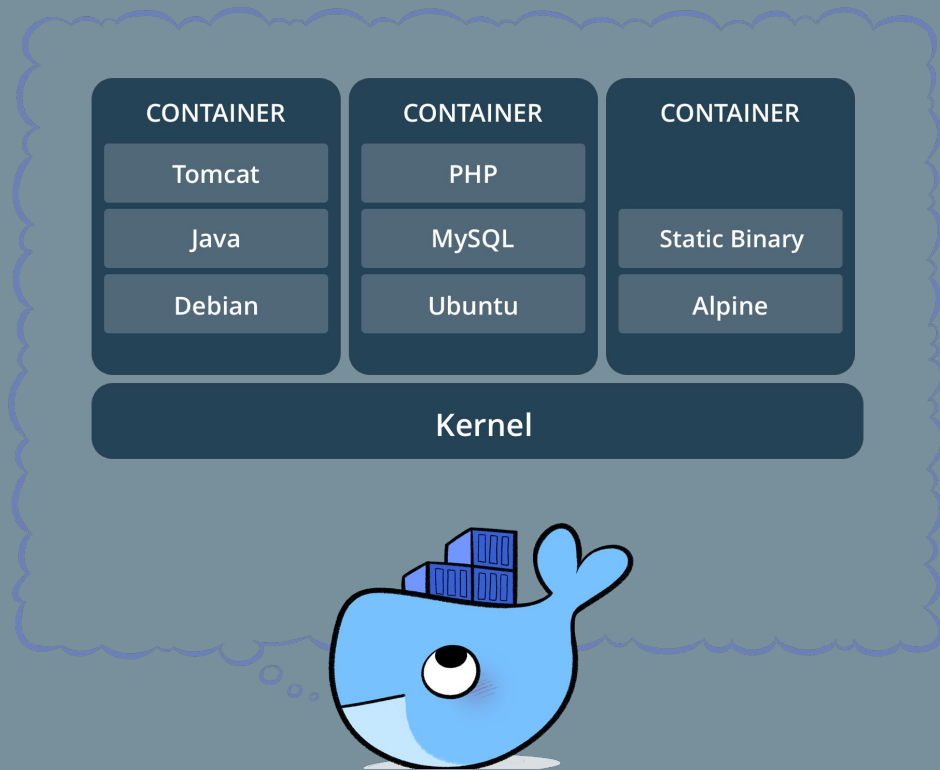
Virtualisation de machines

Credit : veenam.com

Web-scale IT

... - Conteneurisation

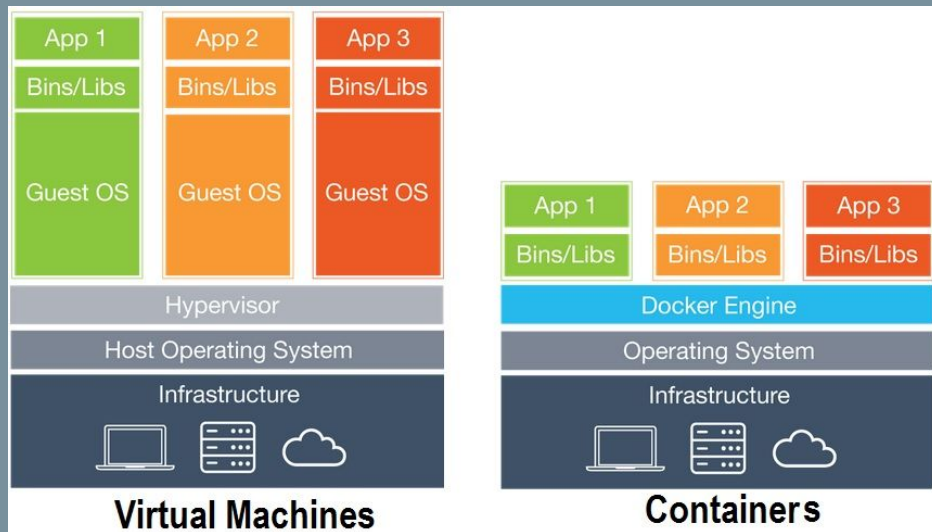
Empaquetage d'une application et toutes ses dépendances (code exécutable, bibliothèques, runtime, configuration...) **dans une image légère, isolée**, qui pourra être exécuté par la technologie de conteneur, sur n'importe quel machine.



Web-scale IT

Cloud Computing - Virtualisation des infrastructures - **Conteneurisation**

Machine Virtuelle	Conteneur
Repose sur Hyperviseur + système d'exploitation	Directement au niveau du système d'exploitation
Empreinte mémoire importante	Pas d'empreinte mémoire
Tps de lancement de l'ordre de quelques minutes	Tps de lancement de l'ordre de quelques secondes
Déploiement parfois pénible. Moins portable	Déploiement plus simple et minimaliste. Plus portable
Impact sur les performances plus important	Impact sur les performances plus faible



Virtualisation versus conteneurisation

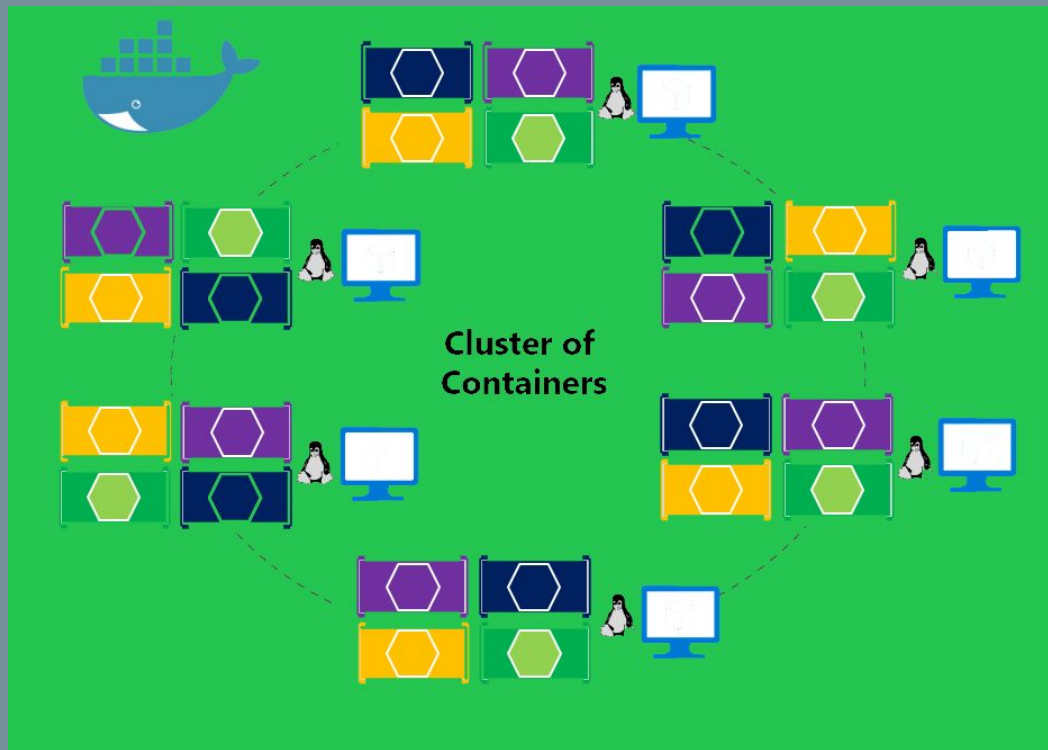
Credit : docker.com

Web-scale IT

... - Conteneurisation

Orchestration de conteneurs

Déploiement de multiples
conteneurs sur de multiples
“machines” pour les applications
“multi-conteneurs” (Microservices)



Amazon ECS



kubernetes



Apache
MESOS



Amazon EKS



OPENSIFT



Google Kubernetes Engine



HashiCorp

Nomad

<https://docs.microsoft.com/fr-fr/dotnet/standard/microservices-architecture/architect-microservice-container-applications/scalable-available-multi-container-microservice-applications>

Web-scale IT

DevOps - automatisation de l'infrastructure - Continuous Delivery

“Apparu vers 2010, le mouvement Devops se caractérise principalement par la promotion de l'automation et du suivi (monitoring) de toutes les étapes de la création d'un logiciel, depuis le développement, l'intégration, les tests, la livraison jusqu'au déploiement, l'exploitation et la maintenance des infrastructures. Les principes Devops soutiennent des cycles de développement plus courts, une augmentation de la fréquence des déploiements et des livraisons continues, pour une meilleure atteinte des objectifs économiques de l'entreprise.” Wikipedia

Web-scale IT

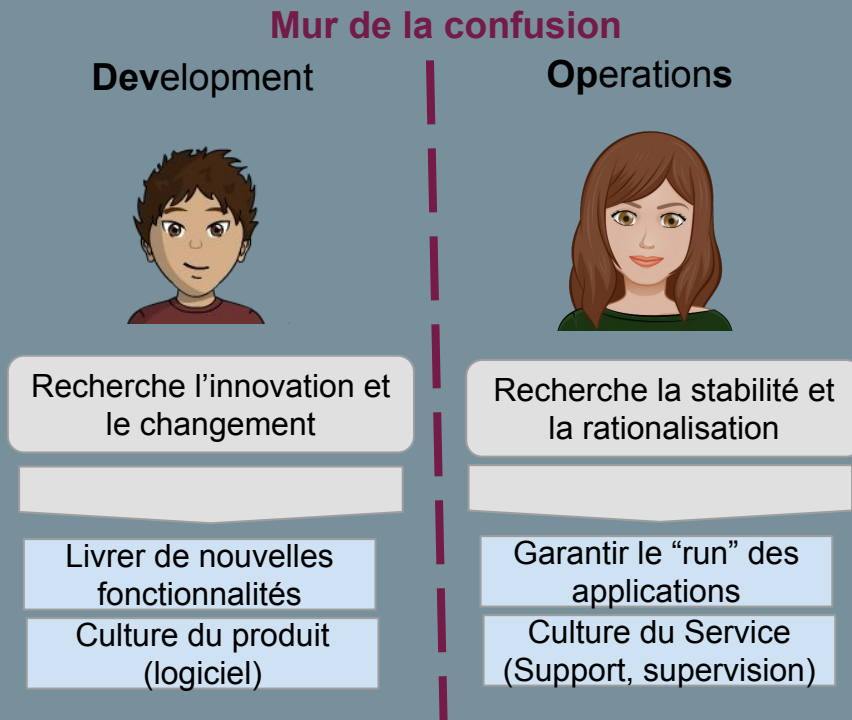
DevOps - automatisation de l'infrastructure - Continuous Delivery

A l'origine du DevOps : des Devs et des Ops (production) non alignés

Objectif commun pour les Dev et Ops : “livrer” un système fonctionnel à l'utilisateur final mais :

des préoccupations distinctes,

une organisation qui ne favorise pas la collaboration et le partage des pratiques (agiles).



Web-scale IT

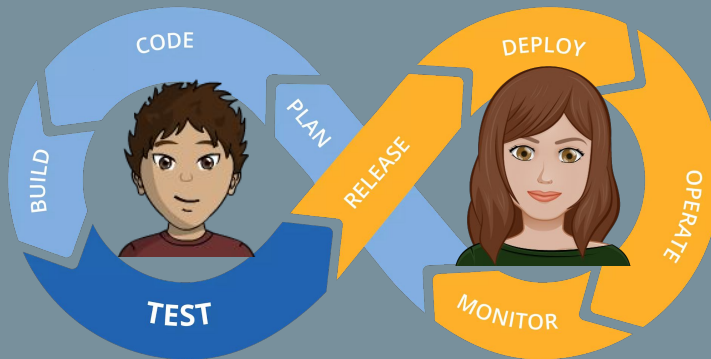
DevOps - automatisation de l'infrastructure - Continuous Delivery

Extension de l'agilité à la production
autour de 3 grands axes :

- Automatisation de l'infrastructure (ou Infrastructure As Code)
- Continuous Delivery
- Culture de la collaboration

Grands objectifs :

- Réduction du **Time To Market**
- Réduction du **Mean Time to Repair**



Web-scale IT

DevOps - automatisation de l'infrastructure - Continuous Delivery

Exigences fonctionnelles
Epics/user stories de fonctionnalités
priorisation du backlog

Tests Fonctionnels
Tests automatisés intégrés dans le build

Utilisation métier de l'application
Remontées de problèmes / bugs
Evolutions fonctionnelles

Analyser

Planifier

Tester

Coder

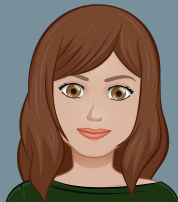
Builder

Déployer

Exigences non-fonctionnelles
Epic/user stories non fonctionnels : Logs,
metrics, Supervision, déploiement

Tests non-fonctionnels
Performances, failover, etc
Continuous delivery

Production
Gestion des incidents - troubleshooting
Problèmes techniques (performance,...)



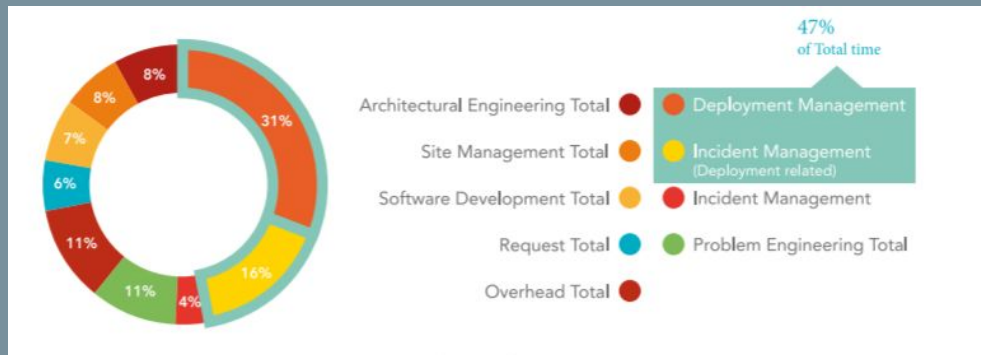
Web-scale IT

DevOps - **automatisation de l'infrastructure** - Continuous Delivery

Un des principes clé du DevOps - plutôt - côté Ops

“automatiser les phases d’approvisionnement et de mise à disposition des environnements, allant de l’instanciation de machines virtuelles au déploiement applicatif, en passant par la configuration système.” Les Géants du Web - Octo Technology

la moitié du temps de la production consommée par le déploiement ou des problèmes liés au déploiement.



Web-scale IT

DevOps - **automatisation de l'infrastructure** - Continuous Delivery

Objectifs/Avantages



CONFIGURATION DE L'INFRASTRUCTURE

Améliorez les performances et la conformité de l'infrastructure dans son ensemble.

- » Assurez la cohérence des configurations.
- » Identifiez et maintenez les niveaux de correctif appropriés.
- » Assurez la sécurité, le contrôle des modifications et le respect des exigences en matière de sécurité.



TÂCHES D'EXPLOITATION ET PROCESSUS

Améliorez l'efficacité et la cohérence.

- » Automatisez les tâches manuelles telles que la configuration, le déploiement, l'intégration et la migration.
- » Proposez des fonctionnalités en libre-service.
- » Réduisez le nombre d'erreurs humaines.



PERFORMANCES ET DISPONIBILITÉ

Apprenez à mieux comprendre le fonctionnement de vos charges de travail.

- » Cherchez à comprendre les dépendances.
- » Anticipez les besoins en matière de capacité.
- » Évoluez automatiquement, de manière fluide.
- » Suivez et facturez l'utilisation réelle des ressources informatiques.

Web-scale IT

DevOps - **automatisation de l'infrastructure** - Continuous Delivery

Infrastructure as Code (IaC)

Gestion de l'infrastructure - configurations et approvisionnement (machines virtuelles, réseaux, [SGBD, serveur web] ...) - de **manière programmatique** (du code) **en appliquant les pratiques agiles du développement logiciel** : gestion de versions, tests, itérations, “petits” déploiements...

=> pratique actuelle **GitOps** (2017) : l'infrastructure est décrite dans Git (voir



HashiCorp

Terraform



Red Hat
Ansible



puppet



Pulumi

Web-scale IT

DevOps - automatisation de l'infrastructure - **Continuous Delivery**

Un des principes clé du DevOps - plutôt - côté Dev

Problématique : *“How long would it take your organization to deploy a change that involves just one single line of code ? Do you do this on a repeatable, reliable basis ?”*

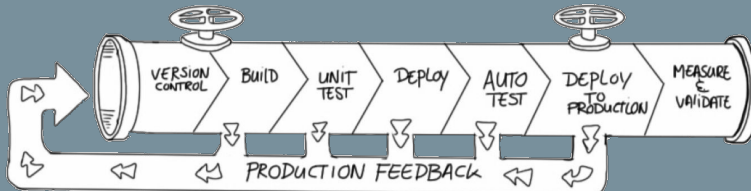
Implementing Lean Software Development - Mary and Tom Poppendieck

Continuous Delivery (ou **livraison continue**) “est une approche d'ingénierie logicielle dans laquelle les équipes produisent des logiciels dans des cycles courts, ce qui permet de le mettre à disposition à n'importe quel moment. Le but est de construire, tester et diffuser un logiciel plus rapidement.” [Wikipedia](#)

Web-scale IT

DevOps - automatisation de l'infrastructure - **Continuous Delivery**

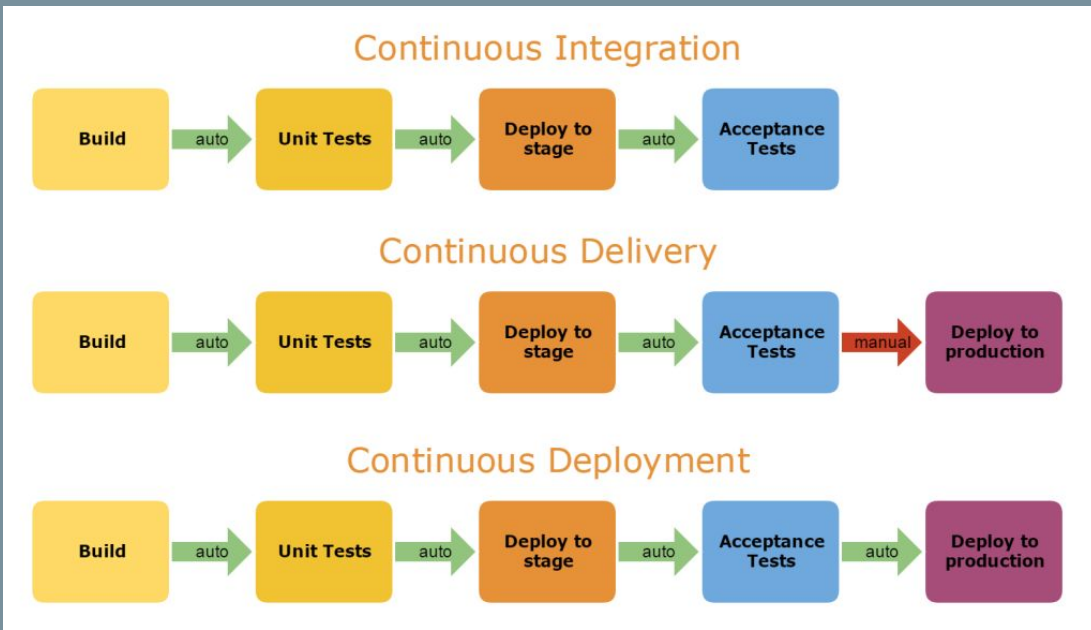
- Principe d'automatisation de la chaîne de construction logicielle (build) et de déploiement
- **Usine de développement** : chaîne de fabrication du logiciel depuis la gestion des sources jusqu'aux déploiement sur les différents environnements (production, pré-production, tests...)



Source Xebia

Web-scale IT

DevOps - automatisation de l'infrastructure - Continuous Delivery



Source Nastel.com



Travis CI



CI/CD



Web-scale IT

DevOps - automatisisation de l'infrastructure - **Continuous Delivery**

Continuous Delivery + IaC

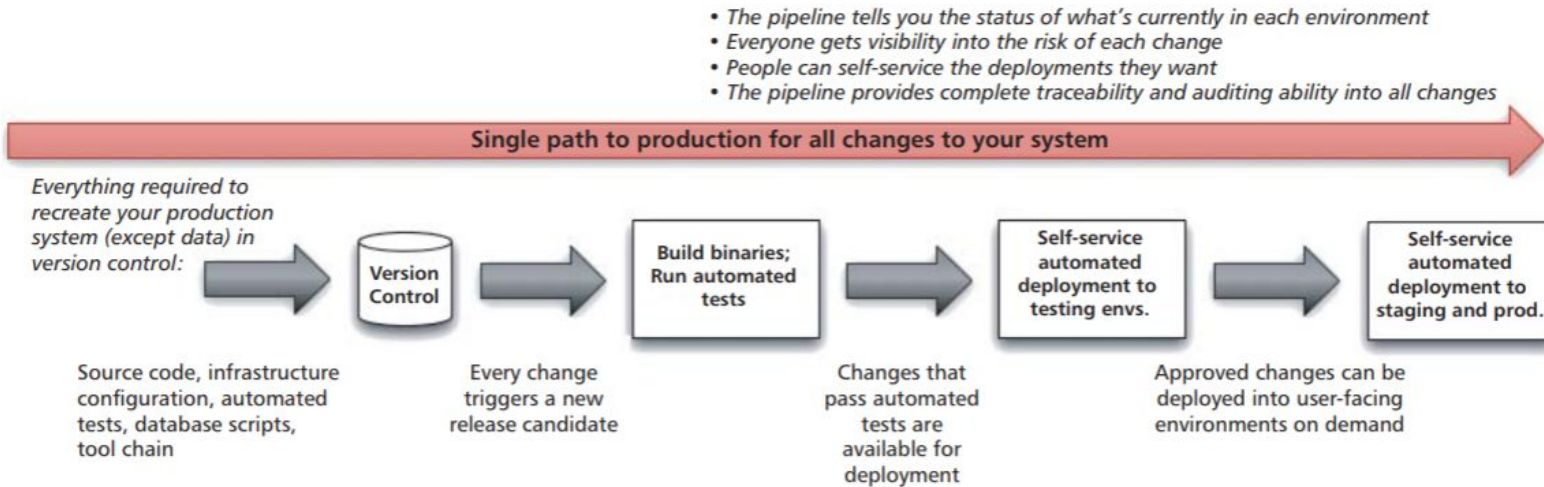
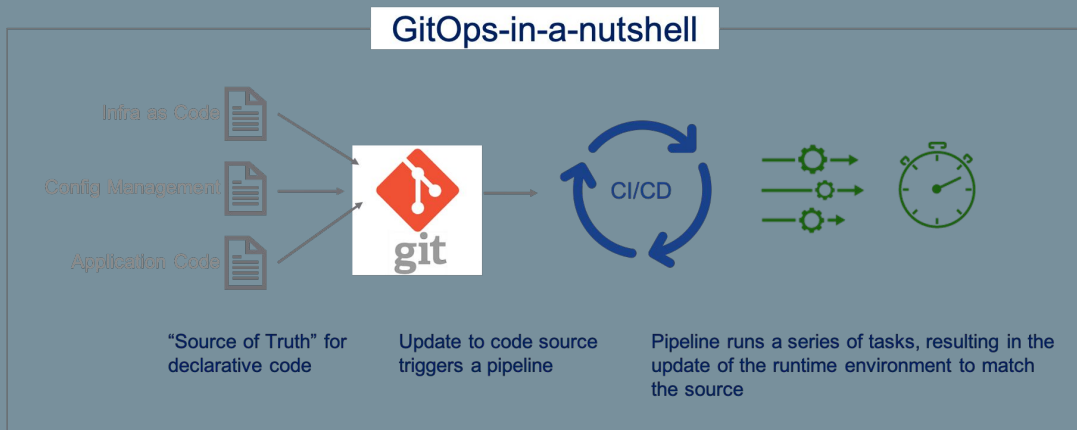


Figure 2 — The deployment pipeline.

Web-scale IT

DevOps - automatisation de l'infrastructure - **Continuous Delivery**

GitOps



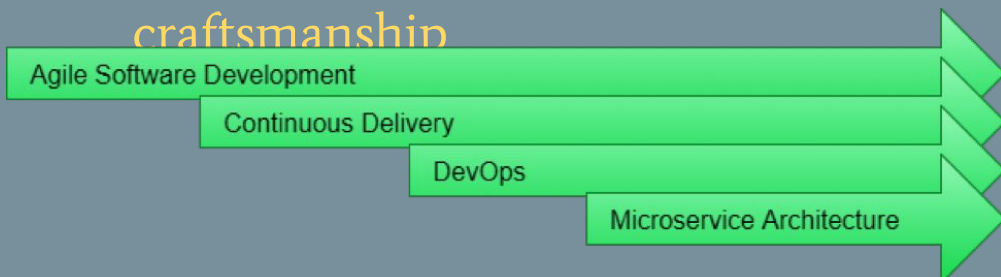
Source : vmware



<https://thenewstack.io/a-look-at-gitops-for-the-modern-enterprise/>

Web-scale IT

- Organisation et processus agiles - Features teams - software craftsmanship



Source : Pyxis Studio

être compétitif = pouvoir livrer des produits/des fonctionnalités rapidement, les améliorer vite et de la bonne façon.

L'agilité = impliquer le client dès le début projet et adopter un processus itératif et incrémental



Web-scale IT

- Organisation et processus agiles - ...

5 Benefits of Agile Project Management



1. Better product quality

Testing is integrated during the cycle, with regular checkups, enabling product changes if needed.



2. Customer Satisfaction

The Product Owner is involved in the process, making customers part of the project.



3. More control on projects

There is more transparency, short meetings and visibility of every step of the project.



4. Risk reduction

The risks of total failure are eliminated as there is constant contact. There is freedom to implement new changes through the process, adapting to the client's needs and preferences.



5. Faster ROI

Thanks to agile's iterations, the features are delivered soon, having benefits from the beginning while the product is still being developed.



37% faster time to market

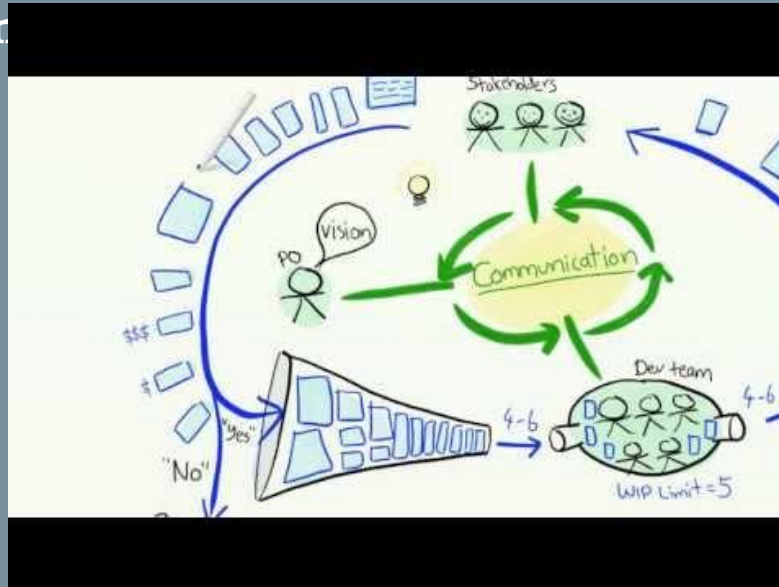
16% higher productivity



Web-scale IT

- **Organisation et processus agiles** - Features teams - software craftsmanship

Gestion agile d'un

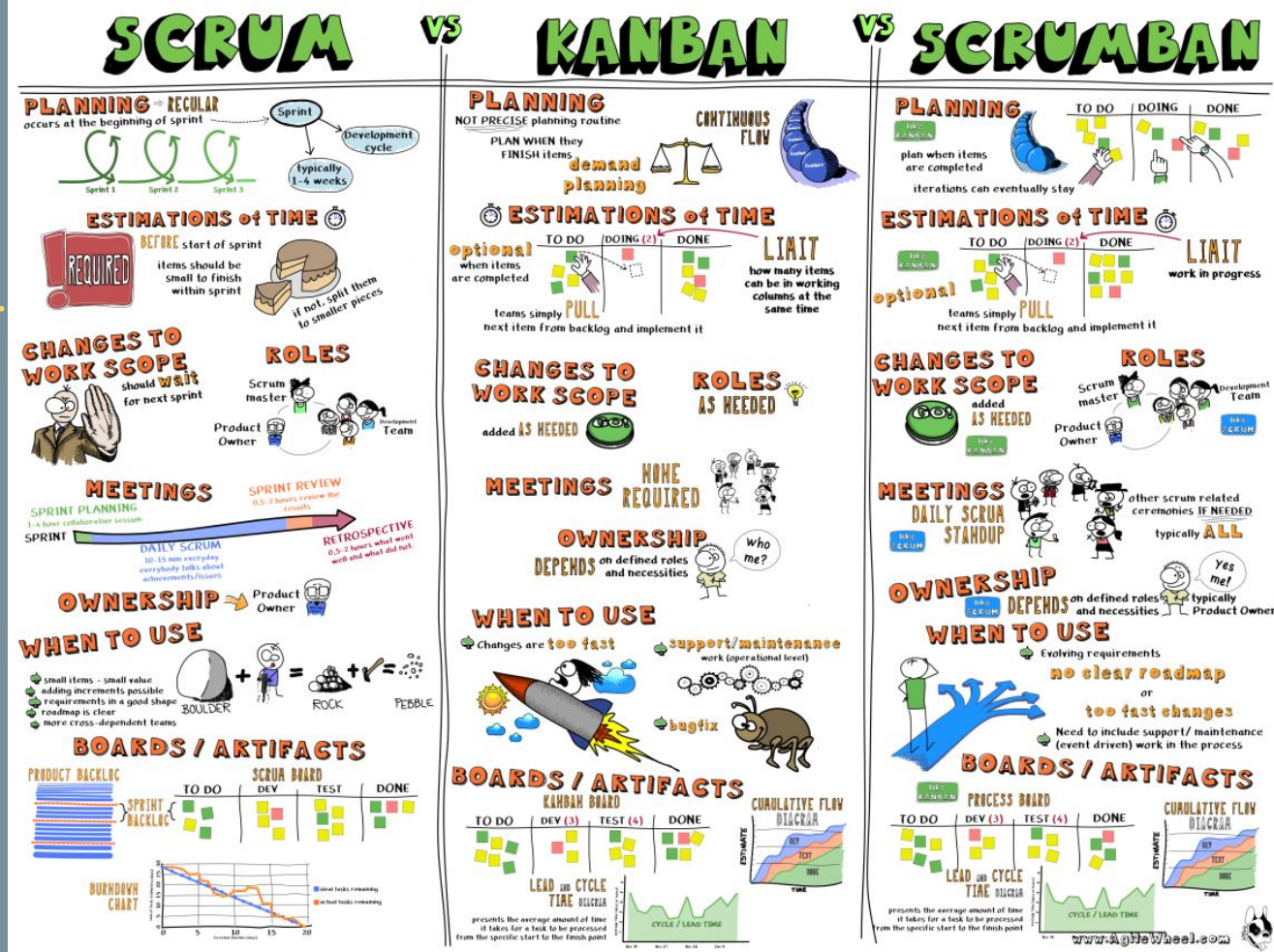


<https://www.youtube.com/watch?v=3qMpB-UH9kA>

Web-scale IT

- Organisation et processus agiles - ...

Source : agilewheel.com



Web-scale IT

- **Organisation et processus agiles** - Features teams - software craftsmanship

Agilité à large échelle

Problématique : proposer un cadre agile à des groupes plus grands qu'une équipe "scrum" (un ensemble d'équipes, une entreprise...) :

- Scaled Agile Framework ([SAFe](#))
- Large-scale Scrum ([LeSS](#))
- [Nexus](#) Framework
- [Scrum@Scale](#)

Web-scale IT

- Organisation et processus agiles - **Features teams** - software

craftsmanship
Problématique
organisationnelle

Organisation classique orientée
projet

Répartition des équipes et des
responsabilités par rôle/fonction :
métier, développement, support,
production... **Ne favorisant pas la
collaboration** : Objectifs locaux
différents (cf. dev et ops)

Méconnaissances des contraintes

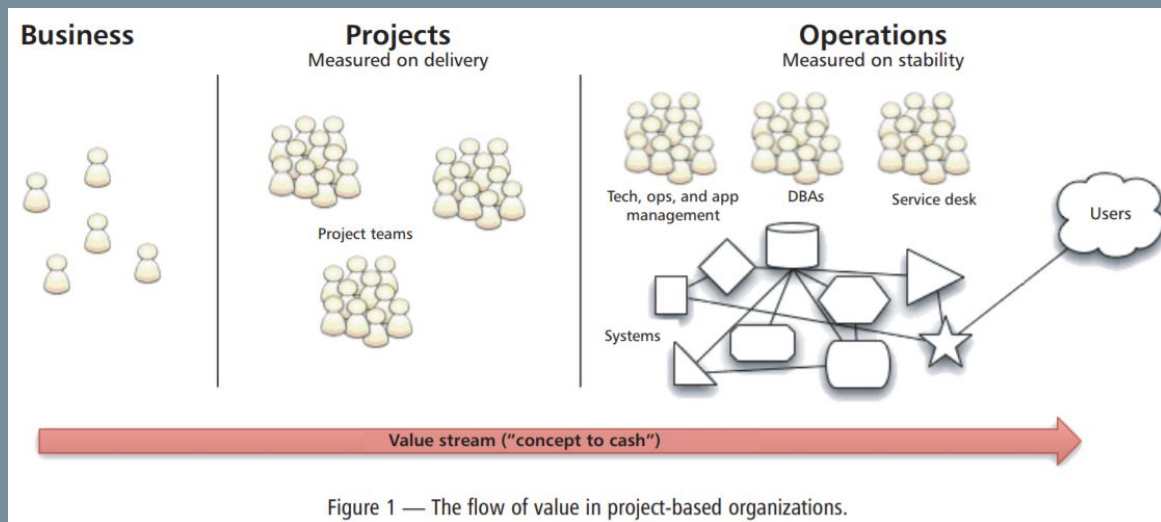


Figure 1 — The flow of value in project-based organizations.

Web-scale IT

- Organisation et processus agiles - **Features teams** - software

craftsmanship Problématique organisationnelle

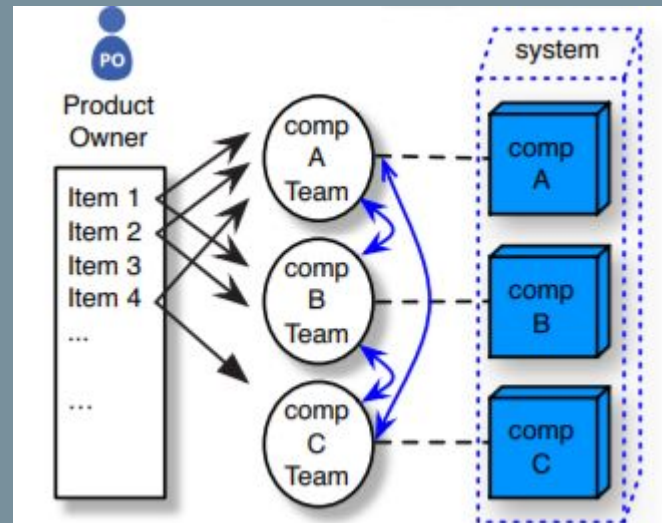
Équipes projets organisées par composant et/couche technologique

Plusieurs équipes pour réaliser une fonctionnalité,
organisation sous-optimale :

Effort supplémentaire de coordination entre équipes

Synchronisation entre équipes : dépendances, livraisons des items en fonction de la charge de travail de chaque équipe

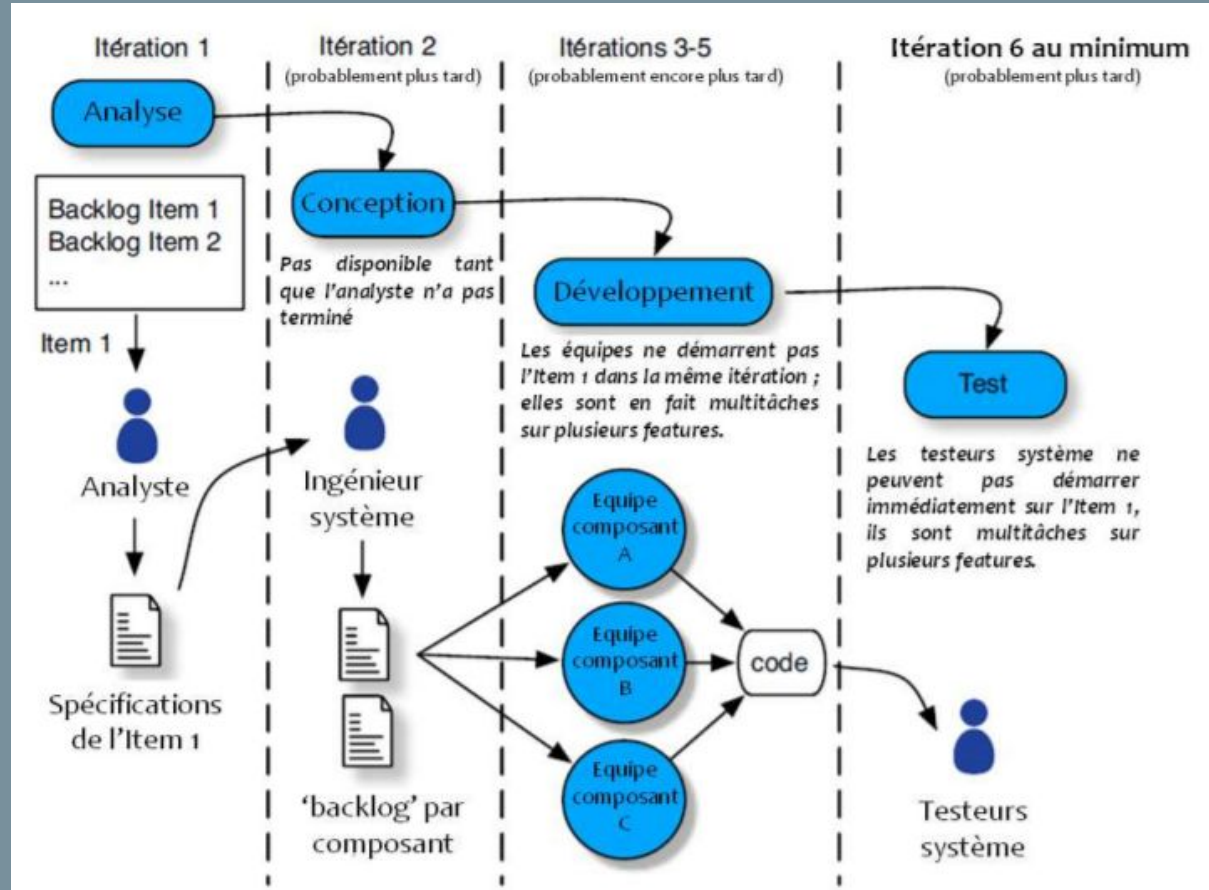
Risque de ne pas livrer la fonctionnalité avec la plus grande valeur en premier



- - **Features teams** - ...

Problématique organisationnelle

Équipes par composant renforce le développement séquentiel (modèle cycle en V ou “cascade”) !



Web-scale IT

- Organisation et processus agiles - **Features teams** - software craftsmanship

Organisation orientée produit / service

Équipe multi-disciplinaire responsables du bon fonctionnement du service :

Une équipe est responsable d'un fonctionnalité / service / produit et inclut métier, architectes, développeurs (front-end, back-end...), ... devops..

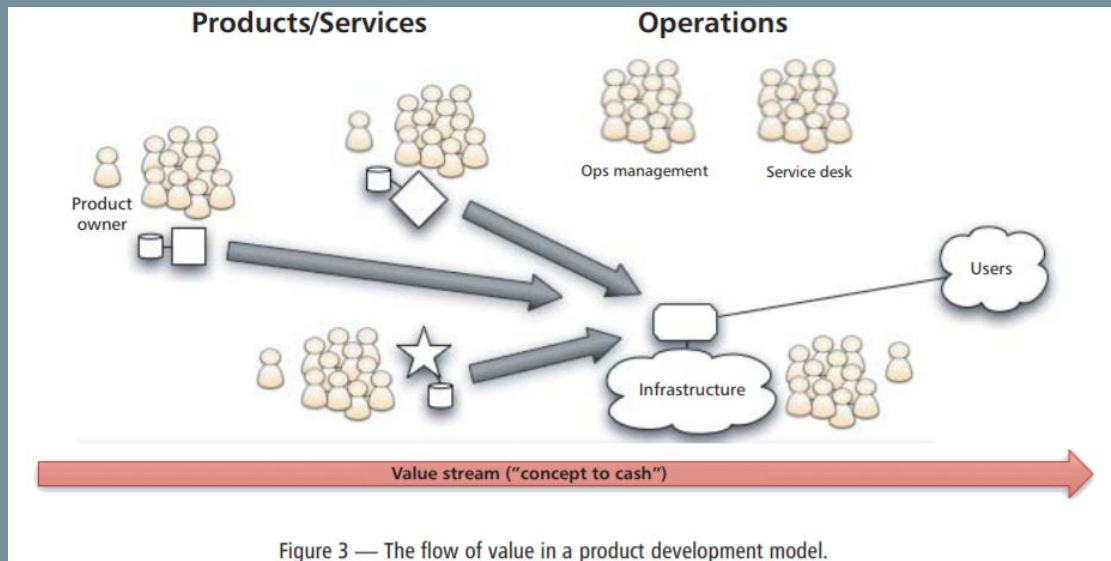


Figure 3 — The flow of value in a product development model.

**"You build it, you run it !" Werner Vogels, CTO
d'Amazon**

Web-scale IT

- Organisation et processus agiles - **Features teams** - software craftsmanship

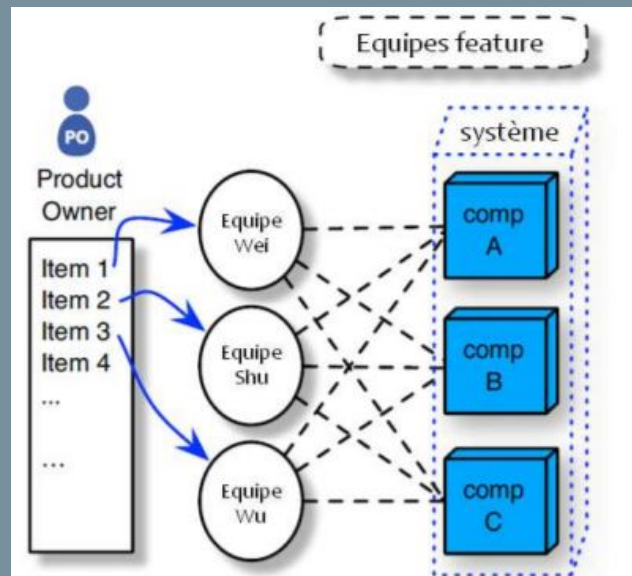
Feature Teams

Organisation optimale pour livrer la valeur métier

- Chaque équipe travaille sur un sous-ensemble fonctionnel cohérent – indépendamment des technologies et des rôles
- Elle est capable de livrer un item du backlog à *tout moment*
- Elle a un peu de dépendances avec les autres équipes

Contraintes/difficultés

- La définition des périmètres
- effort d'apprentissage, périmètre plus large
- Besoin d'intégration régulier
- Communauté pour la cohérence / l'amélioration des pratiques



Source : <http://www.featureteams.org/>

Web-scale IT

- Organisation et processus agiles - **Features teams** - software

craftsmanship
2 Pizza Teams ou la règle des 2 pizzas

"If you can't feed a team with two pizza's, it's too large." - Jeff Bezos, PDG d'Amazon

Taille d'une équipe doit être comprise entre 5 et 12 membres pour être efficace

Signal d'alerte si la taille de l'équipe dépasse 15 membres

Web-scale IT

- Organisation et processus agiles - Features teams - **software craftsmanship**

“mouvement qui prône le côté artisanal du développement logiciel [...], autrement dit, il ne suffit pas qu'un logiciel soit fonctionnel, mais il faut qu'il soit bien conçu. L'idée principale est de garantir la fiabilité et la maintenabilité des applications [...] Le software craftsmanship et l'agilité sont alors complémentaires, car là où l'agilité se limite à la souplesse des cycles de développement, le software craftsmanship s'étend sur la façon même dont est conçu et écrit le code.” Wikipedia

Manifeste de l'artisanat logiciel (2009) <http://manifesto.softwarecraftsmanship.org/#/fr-fr>

Mouvement né en réponse à la tendance, des années 2000, à l'externalisation des développements et à la banalisation du métier de développeur (simple producteur de lignes de code)

Web-scale IT

- Organisation et processus agiles - Features teams - **software**

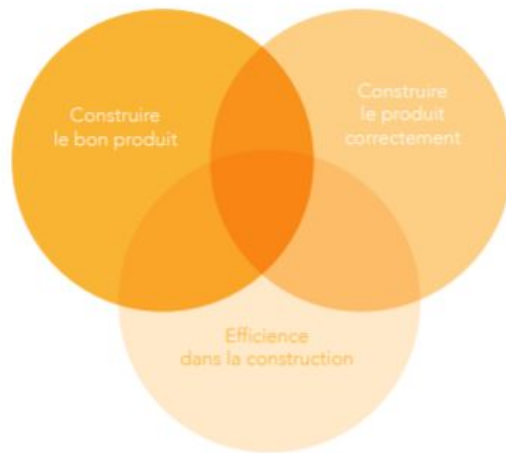
craftsmanship
Une culture de la qualité logicielle

Construire le bon produit , i.e. qui apporte de la valeur et répond aux exigences des utilisateurs : méthodes agiles, MVP..

Construire le produit correctement , grâce à une qualité technique et une maintenabilité suffisante, en réponse des exigences métiers en permanente évolution... : **Software craftsmanship**

Efficient dans la construction : aboutir à de bons résultats avec le minimum de dépense et d'effort : méthodes, organisation, Software craftsmanship

Trois enjeux de construction d'un produit de qualité



Web-scale IT

- Organisation et processus agiles - Features teams - **software**

Pratiques de l'artisan ~~codeur~~ ^{craftsmanship} (inspiré de <https://blog.versionone.com/building-software-craftsmen/>)

Artisanat du Code : utilisation d'un ou plusieurs langages afin d'écriture du code bien conçu avec une préoccupation forte pour la clarté et la lisibilité

Applications des principes de conception : comprendre et appliquer les principes de tels que SOLID, design patterns, refactoring...

Outils et technologies : maîtrise des outils et technologies pour la bonne pratique des activités de développement tels que le refactoring, les tests, la revue de code....

Habitudes de travail : processus d'amélioration continu en tant qu'artisan du code, (auto-)formation, collaboration, (nouvelles) pratiques agiles (TDD, livraison continue...)

Web-scale IT

- Organisation et processus agiles - Features teams - **software**

3 Livres de référence

The Pragmatic Programmer: From Journeyman to Master. Andrew Hunt. Addison Wesley

The Software Craftsman: Professionalism, Pragmatism, Pride. Sandro Mancuso. Prentice Hall

Clean Code: A Handbook of Agile Software Craftsmanship. Robert C. Martin. Prentice Hall

Web-scale IT

- **Passage à l'échelle (*scalabilité*)** - Design for failure

*“La scalabilité (ou passage à l'échelle) désigne la capacité d'un produit à s'adapter à un changement d'ordre de grandeur de la demande (montée en charge), en particulier sa **capacité à maintenir ses fonctionnalités et ses performances en cas de forte demande**”*

Wikipedia

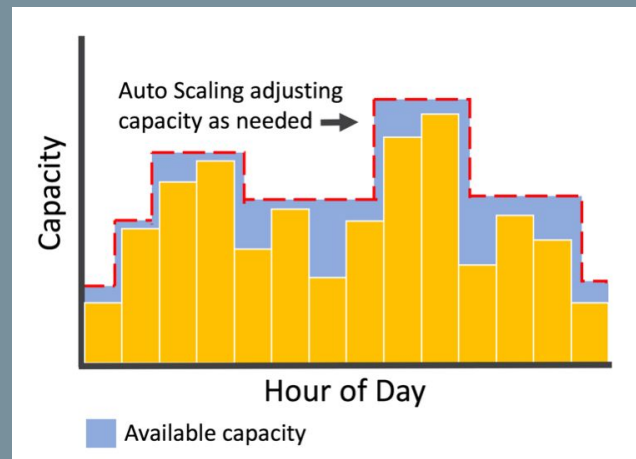
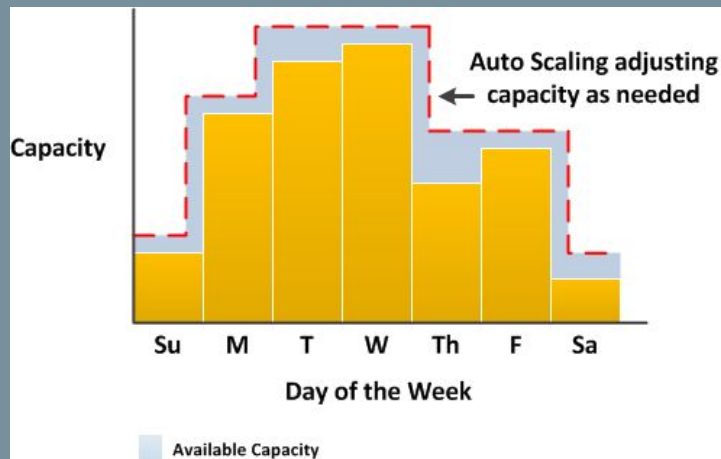
Ajout de la dimension coût : le coût de la requête supplémentaire est a minima identique au coût de la requête précédente

Enjeux majeurs des applications web : ajustement dynamique et transparente des ressources

Web-scale IT

- Passage à l'échelle (*scalabilité*) - Design for failure

Infrastructure élastique (auto-scaling)



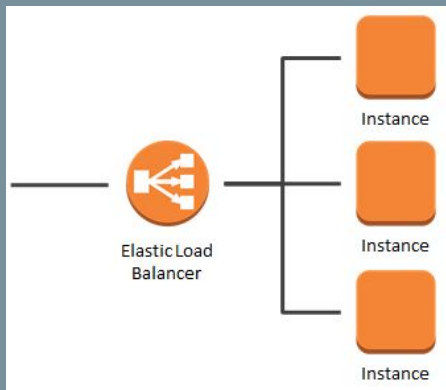
Source : <https://docs.aws.amazon.com/>

Web-scale IT

- Passage à l'échelle (*scalabilité*) - Design for failure

Répartition de charge (load-balancing)

Privilégie la **scalabilité horizontale** : ajout de “serveurs” afin de répondre à la montée en charge. Répartition de la charge effectuée par un **load-balancer**



Source : <https://docs.aws.amazon.com/>

Web-scale IT

- Passage à l'échelle (*scalabilité*) - **Design for failure**

Principes d'architecture technique selon lesquels une application/un système doit être en mesure de supporter une panne (composant logiciel, matériel ou réseau sous-jacent...)

***“Everything fails all the time”* Werner Vogels, CTO d'Amazon**

“The best way to avoid failure is to fail constantly” 5 Lessons We've Learned Using AWS - Netflix

Changement de paradigme, on passe de :

- La prévention de la défaillance et de la panne à elles font partie de l'exploitation
- La crainte d'une défaillance, de crise et réaction à un processus d'analyse et d'amélioration continue

Web-scale IT

- Passage à l'échelle (*scalabilité*) - **Design for failure**

Principes

Redondances d'infrastructures virtualisées avec bascule et déploiement automatisés

Conception des applications de telle sorte qu'elle continuent de fonctionner malgré des défaillances des systèmes ou des applications connectées, avec dégradation potentielle du service (performances acceptables, fonctionnalités désactivées...)

Utilisation des outils de tests automatisés générant des pannes aléatoires en production (arrêt de VM, latence réseau, partie de l'infrastructure...) : [Simian Army](#) - [Chaos Monkey](#) (Netflix)

[Chaos Engineering](#) :

“There's no better way to test something than to break it” CTO of [Gremlin](#)

Web-scale IT

- Passage à l'échelle (*scalabilité*) - **Design for failure**

Principes

Redondances d'infrastructures virtualisées avec bascule et déploiement automatisés

Conception des applications de telle sorte qu'elle continuent de fonctionner malgré des défaillances des systèmes ou des applications connectées, avec dégradation potentielle du service (performances acceptables, fonctionnalités désactivées...)

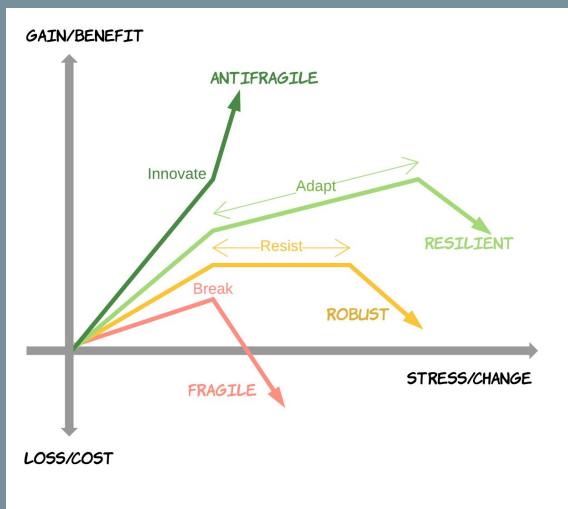
Utilisation des outils de tests automatisés générant des pannes aléatoires en production (arrêt de VM, latence réseau, partie de l'infrastructure...) : [Simian Army](#) - [Chaos Monkey](#) (Netflix)

[Chaos Engineering](#) :

“There's no better way to test something than to break it” CTO of [Gremlin](#)

Web-scale IT

- Passage à l'échelle (*scalabilité*) - Design for failure



<https://developers.redhat.com/blog/2016/07/20/from-fragile-to-antifragile-software>

Niveaux	Definition
Système fragile	Le système fonctionne assez bien et fournit le service attendu aux clients lorsqu'il est utilisé dans un environnement stable et non changeant. La stabilité du système est plus menacée lorsqu'il est soumis à des contraintes et à des changements supplémentaires.
Système robuste	Le système est capable de gérer les erreurs identifiées/communes (entrées erronées, erreurs de fichiers, données incohérentes) et de faire face à certains changements et stress.
Système résistant (resilient)	le système peut gérer plus de stress et de changements, car il a été conçu en tenant compte du stress et des changements.
Système anti-fragile	Résistance au chaos : le système est capable de résister à des conditions turbulentes et inattendues. Les équipes expérimentent les échecs sur le système logiciel en production afin de renforcer la confiance. construit pour le changement : le système est construit en tenant compte du changement.